

Module 3 - Stochastic Gradient Descent and Optimizers

Video Transcripts

Video 1: Gradient Descent (05:39)

We've seen how gradient-descent-based training of neural networks is done in principle. We'll now look at some practical aspects of gradient-descent-based training. To recap: Neural networks can be trained by minimizing a loss function. The loss we minimize, through gradient descent, quantifies the average discrepancy between the target and actual outputs of the network for a set of training instances. As part of the computation, we must compute the average loss over the entire training set. This gives us a way to make the process more efficient.

Let us recall our training formulation — we want to train our network to approximate a function based on only a small number of training instances for which the function value is known. We train the network to compute the correct output at these points. That is, we want to ensure that the function our network computes passes through the training instances and, by doing so, properly captures the rest of the function. To learn the input-output relationship, we use gradient descent. We first initialize our network, and this yields an initial function. The gradient descent update incrementally adjust the network parameters to try to reduce the average error over all the training points.

This process will try to simultaneously correct the error at all of these training points. It will try to push up the network function at training instances where it is below the target function and to push down the network function at instances where it is too high, all at the same time. The updated parameters will give us a new function which may still be off. The next iteration will try to correct the new error at all the training instances. The process continues, trying to correct the error at every training point in each iteration. This process, where we consider the average error over all training instances to update parameters, is called a batch update. We update the parameters to try to correct the errors over the entire batch of training data. The problem with this approach is that before each update step, we must first process the entire training set, which could have millions or even billions of data points.

Here's the pseudocode for batch updates. We are given our training set and we initialize the network parameters. Then within each iteration, we go over the entire training set. In each pass through the training set, for each layer of the network, we compute and aggregate the loss derivative for the parameters of that layer over all the instances in the training set. After the entire training set is processed, we update the parameters with the average gradient over the training set. We do this until the loss has converged.

There is an alternative approach to this: Instead of trying to adjust the function at all training points at once, we will consider one training point at a time and adjust the network parameters to reduce the error at that point. We then select another instance and adjust the parameters to reduce the error at that point. This continues for another point. We continue this process one

instance at a time. At each update, we will keep the actual update small. Eventually, when we have processed every instance, we would have adjusted the function for all of them. Except, now, instead of making just a single update after processing all T training points, we would have made T updates to the function, one after each point. Even though the individual updates are smaller, the net effect of these T updates can be more than what we might get with a single batch update. This approach, where we update the function after each training instance, is what we call incremental update. We will generally make multiple passes over the training data and within each pass, we will update the parameters after each training instance. That is the entire incremental update process is repeated.

We call a single pass over the entire set of training data and epoch. Within each epoch, we have a total of T training samples. We will make T updates to the parameters. Here's the pseudocode for the incremental updates. Within each iteration, as we loop through the training instances, we immediately update the network parameters after each training instance. This outer loop, which represents epochs, updates all T training instances and continues until the loss converges.

Video 2: Stochastic Gradient Descent (03:20)

Let us continue on the topic of incremental updates. To recap: conventional gradient descent updates the parameters using the average gradient over the entire training set. This requires us to process the entire training data before each update, which can be very expensive. These are called batch updates. In incremental updates, we perform gradient descent updates after each training instance. Individual updates are smaller than in batch updates but, since we get many more updates within each pass through the data, this can actually be more efficient than batch updates. However, simply looping through the training samples in the same order every time can result in cyclic behavior.

For example, let's say we have training samples distributed as shown here, and our function is restricted by the model to be of the shape shown by the red curve, and we are trying to approximate the blue dotted line. If we go through the data in sequential order from left to right, the function will first get pushed down to reduce the error at the points on the left but will end up rising on the right. When we process the data to the right, the function will get pushed down at those points. This will push the function back up on the left-hand side.

If we continue looping through the data in the same order, the behavior will repeat. To prevent this cyclic behavior, we must go through the data in random order, to get more convergent behavior. To do this, we randomly select one instance and adjust the function for that instance. This would move the function only at that point. Then we randomly select another point and adjust the function at that instance. This process continues for randomly selected instances. This process of randomly selecting and updating training instances prevents the cyclic behavior and results in a properly updated function. This modified process is called stochastic gradient descent. "Stochastic" because we randomly select the training instances, and "gradient

descent" because we adjust the parameters with the gradient of the divergence for that instance.

The story so far is that incremental updates can yield more updates per pass through the data, but repeatedly processing the data in the same order can cause cyclic and non-convergent behavior. The order of presentation of inputs must be randomized between epochs. Incremental updates with randomized presentation of inputs is called stochastic gradient descent or SGD.

Video 3: Mini-Batch Descent (04:55)

We've compared and contrasted batch and stochastic gradient updates. Let's now look at an intermediate solution that is a bit of the best of both worlds. Because SGD uses the gradient from only one sample at a time to update models, there is a large variance between runs, and the resulting models could be worse than for batch updates. On the other hand, SGD provides much quicker updates and takes less time to converge. We get a good intermediate solution using what are called mini-batch updates. Instead of either updating the function at all training points simultaneously, or updating it at only one point, we will sample a small random subset of our training data, which we call a minibatch and update the model there.

For example, we might first randomly choose and adjust the function at these three training points. Again, remember that we are trying to learn the function that passes through the points. The other curve is the function currently modeled by the network. This adjustment will push the function up where the value of the network output at the training sample is below the target and down where it is above the target function. Now we choose a second, different, set of points and adjust the updated function at those points. We continue to repeat this process until we cover the entire training set and update the model everywhere. Like with SGD, we use small updates but, here, the updates are made within each minibatch. As you might suspect, this process requires randomness to work properly.

We can make multiple passes through the training data but the manner in which we choose the subsets must differ in each pass. Here is a pseudocode for mini-batch update. This is very similar to SGD where we repeatedly loop over randomly shuffled training data. Each iteration, however, now operates over minibatches of instances. The variable b is the mini-batch size. The derivatives are averaged over the minibatch before we update the models. Again, the learning rate must shrink with each iteration. Returning to our K-means example, we now see that mini-batch descent converges quickly like SGD but gives us low error and low variance like full-batch training. Therefore, when we train neural networks, we typically use mini-batch updates.

One issue is how to quantify the current state of the training which is generally defined in terms of the overall training loss on the entire training data. That's what these figures actually show. After each update, the total loss on the entire training data was measured with the

updated model. In practice, it won't be feasible or sensible to measure the overall training loss on the entire training data after each update. Instead, we typically estimate network performance as the loss or classification error on a held-out set of data. Alternately we could simply report the loss on the current minibatch, or a running average of current and past minibatch losses.

The success of the mini-batch method depends on mini-batch size, the learning rate, and the learning rate schedule which describes the learning rate values and how they change over iterations. The mini-batch size is a hyperparameter that is empirically optimized. The minibatches should be large enough to have low variance, but small enough to get the maximum number of updates in each epoch. The learning rate must satisfy the same criteria as with SGD. The simplest popular approach to set the learning rate schedule is to fix the learning rate until the error plateaus and then reduce it by a fixed factor such as 10.

We have just learned that mini-batch updates operate on batches of instances at a time and produce estimates with relatively low variance and fast convergence compared to other models. Mini-batch updates are therefore a popular method for training neural networks.

Video 4: Momentum (04:03)

We have introduced incremental gradient descent methods with SGD and mini-batch descent, but we'll now look at trend algorithms that address some lingering convergence issues. We'll first look at momentum. We saw much earlier that gradient descent updates can have convergence issues, even for batch updates. For incremental updates like SGD or minibatch, the problem gets worse. Each minibatch computes a loss over a different set of data, and so even the loss function we minimize changes from iteration to iteration. This can cause the direction of the updates to vary erratically, affecting convergence.

As an example, let's examine a contour plot of a loss function. The minimum of the function is within the smallest ellipse. The vertical and horizontal directions represent two different parameters of the network. The arrows show updates obtained over a sequence of minibatches. The typical behavior is that they bounce around instead of heading directly towards the minimum as we want. Here's a simple fix. Instead of using the raw gradients obtained from the latest minibatch, we now compute a running average of the gradients obtained over the iterations, and only use the running average at each iteration to update the model. This smoothing approach cancels out oscillations in the directions where they occur and emphasizes directions of consistent updates. This implementation produces better results and leads us to the momentum method.

We'll explore the momentum method with both math and visuals. Consider that the arrow in this figure shows the most recent update, and the dot at the end of it is the current estimate. The current estimate is $W^{(k-1)}$, and we are trying to compute the optimal step, ΔW^k , that will take us to the next estimate, W^k . At each iteration, we first compute their gradient at the

current location and, from it we get a gradient-based step, which is simply the negative of the gradient of the loss, multiplied by the learning rate. Next, we add to it a copy of the previous update scaled by the factor beta. That gives us the new step which brings us to our new estimate shown by the new point. In the next update, we compute the gradient step at the current location, add to it the scaled previous step, and get the new step and estimate. At each iteration, the net update is beta times the previous update – eta times the current gradient. We add this update, ΔW , to the current estimate to get the new estimate.

A typical value of beta is 0.9. This training average has the property of enhancing updates in directions where the gradient retains the same sign but shrinking the steps in directions where the gradient keeps flipping. The overall result is smoother updates. Here is the pseudocode for mini-batch descent with momentum updates. It is the same as before with one change in how the update is performed. First, we compute the running average of the current gradient step and the previous update and then we update the parameters with this running average.

Video 5: Higher-Order Trend Methods: RMSProp (05:21)

We will now look at higher-order trend methods that try to improve on momentum. To recap: Neural networks are trained by minimizing a loss function, which represents the average error on a training set. The loss is minimized with respect to network parameters through gradient descent. Incremental update methods update the network parameters over either individual instances or over minibatches of training data. These methods can be noisy because the gradient used to update the model always chases the error on the latest sample or minibatch. Momentum smooths out the noise in the gradients by using a running average for update. This still ignores another important feature: the actual amount by which the derivative varies.

Let's see an example with two contour plots representing loss surfaces. As before, the horizontal and vertical axes could represent any two network parameters and the arrows represent gradient updates. In both figures, the average gradient over six steps is about the same. That is, the final point is in approximately the same location in both figures. However, in the figure to the left, the updates swing around a lot more than in the figure to the right. There is more total vertical movement in the left figure, though the net movement is the same. It seems the vertical component of the gradients are to be trusted less in the first case than in the second. This suggests that we should reduce the learning rate of the parameter represented by the vertical component. As you probably suspect, this will result in different learning rates for the horizontal and vertical components.

Let's now look at two popular algorithms for this idea. The first is RMSProp, where we quantify the total movement in any direction as the root-mean-squared or the RMS value, of that component of the gradient, and scale down the learning rate by its inverse. To explain it, we will use the following notation. Each of these terms are to be computed independently for each individual parameter: individual connection weights, or biases. The partial derivative of the loss with respect to any individual parameter w is shown as $\text{partial}_w D$. D is the loss for the

minibatch or, if you are performing SGD, the loss on the latest sample. We represent the squared derivative as partial w of D, whole squared. The mean-squared derivative is a running estimate of the average squared derivative.

We will show this is E of partial w of D whole squared. Here is RMSProp. It is a variant on the basic minibatch SGD algorithm. We maintain a running estimate of the mean squared value of derivatives for each parameter. At each iteration, this is computed as an interpolation between the previous estimate of the mean squared derivative, and the square of the current derivative. So, the Kth estimate of the mean-squared derivative is gamma times the (k - 1)th estimate of the mean-squared derivative plus (1 - gamma) times the square of the current derivative, where gamma is between 0 and 1.

Again, this is computed individually for each parameter. The actual update rule now scales the learning rate by the inverse of the root-mean-squared derivative. This epsilon here is added to the term under the square root to prevent division by 0. Learning rates are updated depending on the RMS value. That is, a large RMS value indicates increased component movement, and so we shrink the learning rate in response. A small RMS value indicates reduced component movement, and we increase the learning rate in response.

Here's a pseudocode for minibatch updates with RMSProp. Note that the only difference is that the code now tracks the mean squared value of the derivative by component. It then scales the initial learning rate by the inverse RMS value. The key concept here is that you no longer need a learning rate schedule. The inverse weighting naturally provides you with dynamically updated learning rules. A typical value for the initial learning rate is 0.001. A typical value for gamma, which is the running average parameter is 0.9.

Video 6: Trend Methods: RMSProp with Momentum (ADAM) (04:04)

We now continue our discussion on trend methods with a technique called ADAM which draws its name from adaptive moment estimation. To recap: SGD and minibatch descent updates can be noisy because the gradient always chases the error on the latest sample or minibatch. The momentum methods smooths out the noise in the gradients by using their running averages for updates. RMSprop also adjust the learning rate for each component by the total variation of the derivative to reduce the oscillation in parameter updates. RMSprop only adjust the learning rate, but still uses the original noisy derivative in the actual update.

ADAM uses the smooth running-average derivative along with the adjusted learning rate and effectively combines momentum with RMSprop. To do so, it must maintain two running averages: the main derivative for each parameter m_k and the mean-squared value of derivatives for each parameter v_k . The interpolation factors, delta and gamma, used to compute m_k and v_k , can be different. We have another issue to consider. G and delta lie between 0 and 1 and are generally close to 1, so $1 - \gamma$ and $1 - \delta$ are typically close to

0. As a result, for many of the initial iterations, the m and v terms stay close to their initial values which are usually 0.

If we use them directly, the parameter updates would be too small. So, we divide m_k and v_k by $1 - \delta$ to the power k and $1 - \gamma$ to the power k respectively, to get $\hat{m}_k$ and $\hat{v}_k$. The division ensures that, in the initial estimates, $\hat{m}_k$ and $\hat{v}_k$ have reasonable values. After a few steps, δ to the power k and γ to the power k becomes small and the denominator term goes to 1, and it stops being a factor, so this only affects the initial iterations. The actual update rule now uses the adjusted running average derivative $\hat{m}_k$, and scales down the learning rate by the square root of $\hat{v}_k$. ADAM remains very popular. As with RMSprop, once you get the initial η , δ and γ , you don't need to plan learning rate schedules.

The algorithm automatically learns the best learning rate for each parameter. In fact, if the only advanced optimization method you know is ADAM, you will be fine. Typical parameters for RMSprop and ADAM are similar: low values for η and high values for both δ and γ . Other closely related algorithms such as AdaGrad, AdaDelta, and AdaMax have been proposed but, for most practical situations, ADAM is sufficient.

The story so far is that gradient descent speed can be improved through incremental updates. For convergence, we use randomized input order and a shrinking learning rate. In SGD we update the model after each single input. In mini-batch descent, which can outperform SGD, we update the model after minibatches of inputs. Convergence can be improved further still using smoothed updates and adaptive learning rates, using methods such as RMSprop and ADAM.

Video 7: Batch Normalization Training (11:46)

Let's now look at how we train and perform inference with batch normalization. To recap: Minibatches can have statistical shifts with respect to one another. We can correct this by re-shifting all minibatches to a common location. At each neuron, we first center each minibatch by shifting it to the origin, then we shift the centered minibatch to a new location. This operation is performed independently for the affine value at every neuron, and it introduces two parameters: the scaling and shift for the minibatch, which are different at each neuron. To account for this process, we must modify training to learn the new position parameters, and inference.

Let's first consider training which requires a forward pass that incorporates batch normalization and a backward pass to compute derivatives to learn the parameters. Note that all of the batch normalization operation shown here are at the level of individual neurons in the net. At each neuron, a vector of inputs goes in, a scalar affine value z is computed, and then we put it through an activation.

Batch normalization is a step between the computation of z and the application of the activation. When we perform BN, the affine term z does not include a bias, because the bias term would get canceled out by the centering operation. Here are the operations we perform over a minibatch of input, with BN, at each neuron. We first compute the affine term at the neuron for all the inputs in the minibatch. x_1 through x_B are the B input vectors to the neuron from the minibatch, these may have come, for instance, from the previous layer in the net. W is the row of weights to the neuron. z_1 through z_B are the set of scalar affine values computed for each of the inputs in the minibatch. Each z_i represents the affine value for one instance in the minibatch. Once we have the affine values for the entire minibatch, we only have to consider those. To center the minibatch, we compute the mean and variance of the z values. The mean is a function of all the z 's in the minibatch. The variance is a function of both the inputs and the mean.

In the first stage of BN, for every input in the minibatch, we center the minibatch at the origin by subtracting the minibatch mean from each z term and dividing it by the square root of the variance. The epsilon is a small number meant to prevent division by 0. This yields the set of centered values labeled u_1 through u_B . They will have zero mean and unit variance. In the second stage of BN, we move all the u 's to a new location by multiplying them by a scaling factor γ and adding a shift β . This two-stage BN process gives us the position-normalized affine values $\hat{z}_1$ through $\hat{z}_B$ for the minibatch.

These processed minibatches can then be put through the activations to obtain the outputs y 's, that will be passed on to the subsequent layers. The BN forward pass, at any neuron that incorporates BN, is part of the regular full forward pass in training. Subsequently, the backward pass through the net computes the derivatives at the neuron. During the backward pass, we can obtain the loss derivatives with respect to the $\hat{z}$ terms, $d\text{Loss}/d\hat{z}_i$, by back propagating error gradients through later layers of the network, including the activation of the neuron, for every instance of the minibatch. This is just the usual backprop. We must then backpropagate those derivatives through the second stage of BN.

The second stage of being computes $\hat{z}$ from γ times u plus β and has two learnable parameters: γ and β . We already have $d\text{loss}$ by $d\hat{z}$. So, from equation for $\hat{z}$ we know for any individual instance, $d\text{Loss}$ by $d\beta$ is equal to $d\text{Loss}$ by $d\hat{z}$ and $d\text{loss}$ by $d\gamma$ is equal to u times $d\text{loss}$ by $d\hat{z}$. Since all instances in the minibatch are transformed using γ and β , the actual derivative of the minibatch loss with respect to these terms would sum over the entire minibatch. We can now take a step back and compute the derivative for any u_i from the minibatch, from the same equation: $d\text{Loss}$ by du is equal to γ times $d\text{Loss}$ by $d\hat{z}$. So, we get $d\text{Loss}$ by du for every input in the minibatch. We must propagate those derivatives further back through the first stage of BN to get the loss derivatives for all the z_i 's in the minibatch. Now, as we see here, each z_i influences every u_i in the minibatch, through μ and γ squared. So, to compute the derivative of the minibatch loss with respect to any z_i , we must sum the partial contributions of all the u terms in the minibatch. We already have $d\text{Loss}$ over du_j . So, we only need to figure out the derivative of u_j with respect to z_i , for every (i, j) pair. For direct connections, when j equals i , for

example, where j and i are both 1, this derivative is given by the formula given here. We can derive it by considering all the highlighted parts between z_i and u_i .

For cross connections where j is not equal to i , as shown where j is 2 and i is 1, we get a different formula. Putting this all together, we obtained the derivative of the minibatch loss with respect to each of the z_i 's in the minibatch. One thing that emerges from the formula is that, if all instances in a mini batch are identical, or even very similar, $d\text{Loss } du_j$ will be very close for all instances in the minibatch. Then the second term will cancel the first term, and the third term will also go to 0. So, if all the instances in your minibatch were identical or very similar, this derivative would go to 0, and backpropagation would get blocked at the neuron. Batch normalization actually works best when there is diversity among the instances in the minibatch. We have now computed the derivative of the loss with respect to each of the input affine values of the minibatch, at the neuron.

We can backpropagate those derivatives further through the incoming affine function to the rest of the network in the usual manner. To summarize, the derivatives we get for β and γ can be used to update the estimates of β and γ at each neuron. The derivatives for all of the z_i 's the minibatch can be backpropagated further. What we saw so far was about training with BN. The same computations must be followed during inference on test data. During inference however, test data must be considered as isolated instances, and so we cannot compute a minibatch mean μ_B , a variance σ_B^2 , we must get these from the training data. At each neuron, we use the final trained network to compute the average minibatch mean and variance over all training minibatches – I've represented these as μ_{BN} and σ^2_{BN} here and use those in inference.

In particular, for the variance, we like to use the unbiased estimator, rather than the usual variance estimate. This means that, for the final variance that we compute for using inference, we will scale the average variance of all minibatches by B over $B - 1$, where B is the minibatch size. We won't go over why this is the unbiased estimate. This is a standard statistical concept which you can easily look up. We now plug in the μ_{BN} and σ^2_{BN} into the centering formula along with the learned β and γ , to compute $\hat{z}$ from u to perform inference.

Batch normalization is a neuron-wise operation. We don't need to apply to every neuron in the network. We can apply it only to some layers in the net, or even to only some neurons in the network. It generally improves both convergence rate and neural network performance. Empirically, we often find that batch eliminates the need for dropout, a common regularization technique we will learn about later in this module. To get maximum benefit from BN during training, the learning rates must be increased, and the learning rate decay can be faster. As I mentioned earlier, BN works best if the data within each minibatch are sufficiently diverse. So, to be effective, BN needs better randomization of the training data. If all goes well, batch normalization can result in big improvements in network performance. Here are some results on Imagenet from the original batch normalization paper. The X-axis is the iteration number, and the Y axis is classification performance. The authors tested batch normalization variance

and we see that, in every case, the use of BN greatly speeds up training and improves classification performance versus the baseline.

So, our lessons are: Gradient descent can be sped up by incremental updates, such as minibatch descent. Minibatches may have undesired covariate shifts between them. And covariate shift between training minibatches may be handled by batch normalization.

Video 8: Function Underspecification (05:29)

Another very important problem is that of data underspecification. The figures presented so far paint a somewhat unrealistically optimistic picture. Recall that we try to learn an entire function from just a few of its input-output relationships. The reality is, there is no assurance that the network will learn what we want it to learn. Therefore, we need additional constraints, or restrictions.

To illustrate the problem, consider the case of a simple binary classifier working on scalar inputs. The two colors indicate two classes which overlap a bit near the middle. Assume a network with sigmoid activations at the units. We would want it to learn the sigmoid curve from these data. However, the neural net is a universal approximator and can learn pretty much any function. If there are no other constraints, there's nothing stopping it from learning this other curve instead, which in fact fits the training data better than the curve we want to learn. Why, then, is the network able to learn this undesired function? This undesired function has many sharp changes from 0 to 1 which fit the training data, and the network can learn this function because the perceptrons in the network are individually capable of modeling such sharp changes in the output.

Consider a single perceptron with sigmoid activation. The sigmoid is a smooth curve that goes from 0 to 1, but the rapidity with which it rises to 1 can be made to increase arbitrarily by simply scaling up the weights of the inputs. Given two sigmoid functions, the sigmoid for the larger weight increases much more rapidly than for the smaller weight. The network can learn this unwanted function simply by scaling up the weights for the individual neurons so that they model these very steep changes in the output. To prevent this from happening, we have to constrain the weights for each of the neurons to low values, so they are forced to learn the slower-rising response, rather than the steep one. To achieve that, we redefine our loss function. We originally defined the loss as the average divergence over the data. It is a function of network weights. We estimate the weights to minimize the loss. When we want to also impose a requirement that the learned weights must be small, we add an additional weight regularization term to the loss. This is the sum of the squares of all the weights in the network.

We've represented it as a Frobenius norm here. This representation W_F^2 is simply the sum of the squares of all of the entries in W , so it is the sum of the squares of all the weights and biases in every layer. We will now minimize this modified, or regularized loss. The resulting estimate will not only minimize the divergence for the training samples, but also

simultaneously try to keep the weights small. The factor λ is a regularization parameter which decides the relative importance of keeping the weights small. If we increase λ , it will assign greater importance to shrinking the weights. To minimize the new loss function, we must now compute the derivative of this regularized loss for gradient descent. The derivative of the regularized loss is the derivative of the standard loss, plus the derivative of the regularization term, which is simply λ times the transpose of W . The modified gradient update rule now includes an additional term which subtracts η times λ times the current parameter value itself.

This can be rewritten as in the second update equation: The update multiplies the current estimate by δ by applying the gradient correction. δ nominally equals $1 - \eta \lambda$, but more realistically, it is a term between 0 and 1. Weight regularization is thus achieved by scaling down the current estimate by a factor δ before applying the gradient correction. For this reason, it is often also called weight decay. Here is the modified pseudo code for minibatch updates with weight regularization. It is the same as the usual minibatch update code with one minor modification. We now scaled down W_k by δ before applying the gradient step.

The story so far then is that data underspecification can result in over fitted models and must be handled by regularization.

Video 9: Dropout (04:39)

We've seen that regularizing the network by shrinking the network parameters can mitigate data underspecification. Another effective regularization method is dropout. Before we get to dropout, we need to first visit a popular machine learning technique called bagging proposed by Leo Breiman. The idea is to train several classifiers by sampling the training data. Each classifier is built on a different sample set. Test instances are now classified by voting over the outputs of the entire ensemble of the classifiers. Bagging is known to improve performance significantly over training a single classifier from the entire training data.

Now, back to dropout. Dropout is a training method where in each iteration for each input, we randomly turn-off or dropout some of the neurons in the net with some probability $1 - \alpha$. Alternately, we could retain neurons with probability α . So, if we began with this network, as soon as we receive an input, we flip a virtual coin that has probability of heads equal to α for each neuron. And if the coin turns up tails, we switch off the neuron, effectively taking it out of the network. Practically speaking, the virtual coin will be a Bernoulli random number generator with the probability of success equal to α .

Consequently, the actual network that processes an input maybe like this one, which only has a subset of the total neurons in the network, a fraction α of the total neurons in fact, operating on a subset of the input components. We won't explicitly remove the neurons. Instead, we set their outputs to 0, so they no longer contribute to the computations. The key is

that the specific set of units that are switched off will be different for different inputs. Here, we choose different units to switch off for each of the three inputs X_1 , X_2 , and X_3 . Each of the instances then effectively sees a different network. In fact, even for the same input we will have different deactivation patterns in different epochs, so that the input effectively sees different nets each time. During training, gradient descent has only performed over this reduced network.

The statistical justification that is offered for dropout is that it is some form of bagging. If the net has a total of N neurons, each of which could be either on or off, there are 2 to the power N possible sub-networks that we can compose by switching off different subsets of neurons. The argument is that dropout samples over these 2 to the power N possible networks for each input, and the final network we learn effectively averages over all 2 to the power N possible networks. We are getting the benefits of bagging with this approach. Here's how we would implement dropout.

During the forward pass, at each neuron, we will run a Bernoulli random number generator with the probability of success α to compute a mask. If the generator output is 0 , the mask value becomes 0 , effectively switching off the neuron, otherwise the mask value is 1 . We simply multiply the output of each neuron by its mask. The only layer that is not must is the output layer. During the backward pass, we multiply the derivative with respect to each neuron output Y by its mask value, obtained during the forward pass. That is the only change to make. Keep in mind that the switched off Y values or the neuron outputs for switched off neurons, have already been masked in the forward pass and already are 0 .

Video 10: Testing and Implementing Dropout (03:06)

For a network of N neurons, dropout effectively computes an ensemble average, or the expectation over all 2 to the power N networks that could be composed from it. The problem is, for even a small network with only $1,000$ neurons, 2 to the power N networks are impossible to evaluate explicitly. Instead, we say that the expectation of a network of neurons is approximately equal to a network over the expectations of neurons. Here, $E[y_j(k)]$ is the expected output of the j th neuron in the k th layer. This approximation is easily computed. Each neuron actually computes the product of a Bernoulli variable D and the actual activation function, where D decides if the neuron is switched on or off. It has a probability of success α , so the expected output of the neuron is α times the actual activation function. At the time of testing, we use the expected output of the neuron, where we simply scale the output of each neuron by α .

The simplest way to implement this is to multiply the output of each neuron by α , which is the probability of keeping it switched on. Alternately, instead of multiplying the output of each neuron by α , we can pre-multiply the outgoing weights from the neuron by α . So, the weights we use at test time will be α times the weights we have learned from training. Alternately, during training, we can replace the activation of all neurons by α to the power

– 1 times the function sigma. This doesn't affect the dropout procedure itself during training. We'll just use sigma as the activation during testing and not modify the weights.

A number of variations on the dropout idea have been proposed. In Zoneout for RNNs, randomly chosen units remain unchanged across the time transition. DropConnect drops individual connections instead of nodes. Shakeout scales up some randomly selected weights and fixes the remaining weights to a negative constant. In Whiteout, we add or multiply weight-dependent Gaussian noise to the signal on each connection.

To add to a story so far, dropout as a stochastic data or model erasure method that sometimes forces the network to learn more robust models.

Video 11: Heuristics (02:55)

Finally, we look at some tricks to optimize networks. Early stopping is one approach to prevent overfitting where we track the performance of the network on some held-out data and stop training when this begins to degrade. So here, we would stop training at about the iteration corresponding to the second valley in the validation curve. The derivative of the loss will often be too high. This happens when the divergence has a steep slope at some point, and it can prevent your learning from converging.

Gradient clipping sets a ceiling on the derivatives, typically at five. If the derivative exceeds the ceiling, it is set to the ceiling. We often won't have enough training data. We can augment it with distortions of the data we do have, like rotating, stretching, or adding noise to images such that it doesn't change their class. There are other tricks, like normalizing the training data to have zero mean and unit variance. The manner in which we initialize the net is important, and many methods have been proposed like Xavier initialization and Kaiming's initialization.

Here's the overall process for setting up a problem: Obtain training data. Use appropriate representation for inputs and outputs. Choose the network architecture. Choose the appropriate divergence function. Choose regularization. Choose heuristics such as batch norm, dropout, etc. Choose the optimization algorithm such as ADAM. Perform a grid search for hyperparameters such as learning rate, regularization parameter, etc. on held-out data. Train and evaluate periodically on validation data for early stopping, if required.

In closing, we have outlined the process of training neural networks. We went through some history, we looked at a variety of algorithms, learned about gradient descent-based techniques, we saw how to use regularization for generalization, and discussed a number of heuristics. To make it all work so that you can confidently train your networks, you will need lots of practice. Your programming assignments are therefore designed so you can apply what you learned here. In the next module, we will explore convolutional neural networks.